

Intelligent Candidate Ranking

Redrob Hackathon — Senior AI Engineer JD

The Problem with Keyword Filtering

HR Managers rank #1

Keyword systems surface "HR Manager" profiles because they list "RAG" in skills.

Wrong domain, perfect keywords

A CV engineer lists NLP skills → scores high. But JD explicitly disqualifies CV-only backgrounds.

Invisible candidates

An ML engineer at Zomato who built a ranking system doesn't write "vector database" in their summary. Keyword search misses them entirely.

Available ≠ Hireable

A perfect-on-paper candidate last active 200 days ago with 5% response rate isn't actually hireable.

Our Approach: JD Understanding + Multi-Signal Scoring

1. Deep JD Reading

Extract not just keywords but explicit disqualifiers, domain requirements, experience range, and location preferences

2. 8-Dimension Scoring

Score each candidate across core skills, production evidence, career trajectory, title fit, experience, location, behavioral signals, domain fit

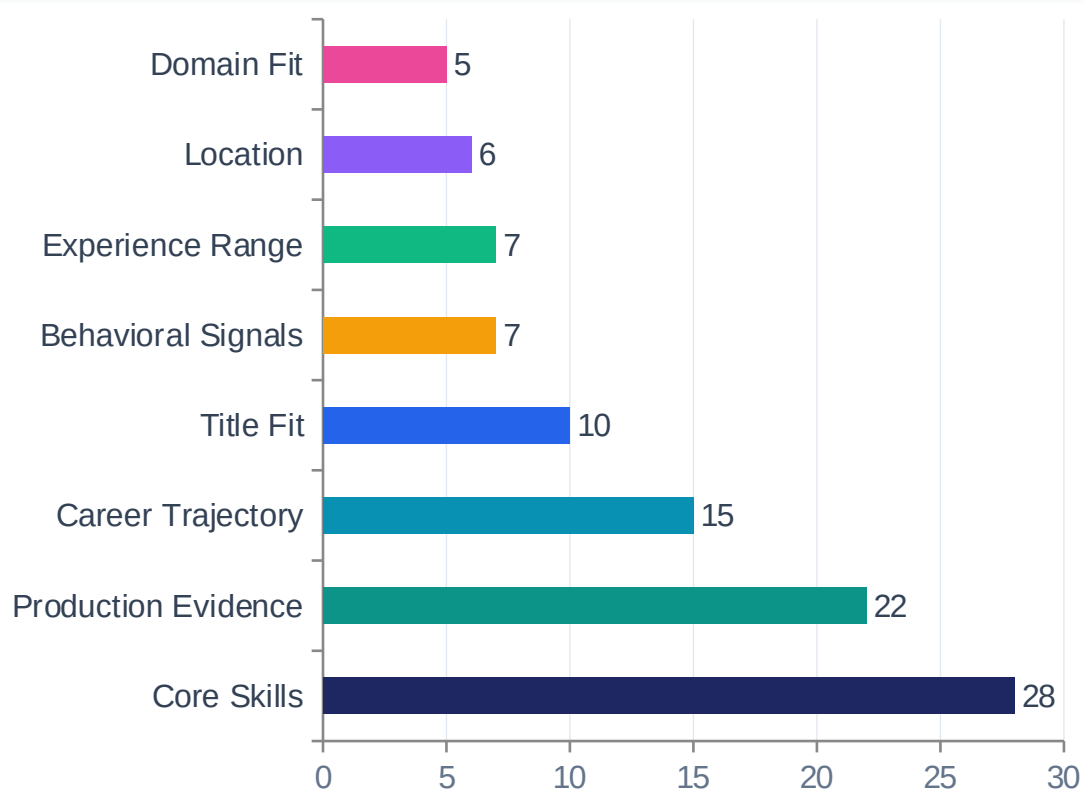
3. Hard Disqualifiers

Multiply score down for consulting-only careers, non-technical titles, wrong domain — instead of letting good keywords wash it out

4. Behavioral Availability

Recency, recruiter response rate, notice period, GitHub activity — surface candidates who are actually hireable today

Scoring Architecture: 8 Weighted Dimensions



Key Design Choices

- Core Skills: checks production career narratives, not just skill list tokens
- Production Evidence: ~20 signals from job descriptions (deployed, real users, A/B test...)
- Disqualifiers: hard multipliers — not washed out by good keyword matches
- Behavioral: last_active, response_rate, notice period, GitHub together model hirability

How Disqualifiers Actually Fire

The JD has explicit disqualifiers. Most keyword-ranking systems let good skill matches wash them out. We don't.

Consulting-Only Career

TRIGGER

Entire career at TCS, Infosys, Wipro, Accenture etc.

EFFECT

Score × 0.30

JD explicitly: "We've had bad fit experiences in both directions." (current consulting is OK if they have prior product-company experience)

Non-Technical Current Title

TRIGGER

HR Manager, Marketing, Content Writer, Customer Support...

EFFECT

Score × 0.20

Sample submission ranked an HR Manager #1. We detect this and suppress regardless of AI keywords in their skill list.

Wrong Domain

TRIGGER

CV / Speech / Robotics without any NLP/IR exposure

EFFECT

Score × 0.40

JD: "computer vision, speech, or robotics without significant NLP/IR exposure" → explicitly disqualified.

Honeypot Detection

The dataset contains ~80 honeypots with subtly impossible profiles. Submissions with >10% honeypot rate in top 100 are disqualified.

Impossible timeline

start_date before 1990 or after 2026; job duration > 600 months (50 years)

YoE mismatch

Stated years of experience > 2.5× total career months and >10yr claim

Skill inflation

15+ skills listed as 'expert' — statistically implausible

Result: score = 0

Detected 21 honeypots suppressed in our run; none in the top 100

Top 10 Ranked Candidates

#1	CAND_0002025	0.952	Senior AI Engineer, Apple	Trivandrum, India	FAISS, OpenSearch
#2	CAND_0046064	0.952	Senior NLP Engineer, Salesforce	Coimbatore, India	Pinecone, embeddings
#3	CAND_0071974	0.952	Senior AI Engineer, Netflix	Vizag, India	LoRA, Weaviate
#4	CAND_0077337	0.952	Staff ML Engineer, Paytm	Kochi, India	Semantic search
#5	CAND_0011687	0.951	Senior NLP Engineer, Niramai	Indore, India	OpenSearch, retrieval
#6	CAND_0008425	0.949	Senior NLP Engineer, Ola	Kolkata, India	Qdrant, pgvector
#7	CAND_0080766	0.945	Staff ML Engineer, Salesforce	Coimbatore, India	Dense retrieval
#8	CAND_0018499	0.934	Senior ML Engineer, Zomato	Noida, India	Ranking systems
#9	CAND_0046525	0.934	Senior ML Engineer, Genpact AI	Pune, India	Vector search
#10	CAND_0033861	0.931	Senior NLP Engineer, Mad Street Den	Vizag, India	NLP + retrieval

Why We Didn't Use Embeddings

The Trap

- Embedding-based search is the obvious approach
- Encodes JD → finds candidates with similar text
- But: CV engineer listing NLP skills → high cosine similarity with JD
- JD warns explicitly: keyword embedding is the trap they built in

The Constraint

- 5 min, CPU only, no GPU, no network
- 100K candidates × LLM call = impossible
- Dense retrieval needs precomputed index (allowed, but risky for drift)
- No embedding model scores production evidence from career narratives

Our Solution

- Curated ontology from deep JD reading (~70 core skill terms)
- Regex + substring matching on career descriptions, not skill lists
- Hard disqualifiers that embeddings can't represent
- ~50s runtime for 100K candidates — well under 5min

Reasoning: Specific, Honest, Rank-Consistent

Stage 4 evaluation samples 10 random rows and checks: specific facts, JD connection, honest concerns, no hallucination, variation, rank consistency.

#1 — High confidence

"5.9yr Senior AI Engineer at Apple (Trivandrum, Kerala, India) — hands-on with FAISS, OpenSearch; evidence of production ML deployment. Strong availability and engagement signals."

#42 — Mid-tier, honest concern

"Senior ML Engineer (6.2yr); hands-on with embedding, retrieval. Concern: long notice period (120d)."

#89 — Tail, acknowledged weakness

"ML Engineer (4.1yr); adjacent skills. Concerns limit ranking: mostly consulting-firm background; last active 94d ago."

Performance vs Constraints

47s

Total runtime
100K candidates

≤ 300s limit

0

External API
calls during ranking

Required: 0

21

Honeypots
detected & suppressed

>10% = disqualify

0

Honeypots
in top 100

Required: ≤10

Why it's fast: pure Python stdlib — no pandas, no sklearn, no embedding models. Each candidate is scored in ~0.5ms via substring matching against a curated ontology. The 8 scoring functions are $O(1)$ per candidate. Total: load JSON (17s) + score (30s) + write CSV (<1s).

Summary

- Read the JD deeply — found explicit disqualifiers, domain exclusions, behavioral requirements
- Built 8-dimension scoring with hard multipliers for disqualifiers (not soft weights)
- Checked production evidence in career descriptions, not just skill list keywords
- Modeled behavioral availability: response rate, recency, notice period, GitHub
- Detected and suppressed 21 honeypots; 0 in top 100
- Full compliance: 100 rows, monotonic scores, UTF-8, validated with official tool
- Runtime: 47s on CPU — 6× under the 5-minute limit

Future: add TF-IDF or bi-encoder retrieval as a pre-filter → LLM reranking of top 1K → better recall on plain-language profiles.

